GUIA TÉCNICO / 01 / 06 NOV 2026

Processador distribuído de transações de apostas

NestJS + TypeScript + Bun | PostgreSQL + SQS/LocalStack

O que o sistema resolve

Recebe apostas, ganhos, perdas e reversões por HTTP ou SQS. Aplica cada efeito financeiro de forma consistente, mesmo com mensagens repetidas, concorrência e reinícios. O saldo pode ser reconstruído pelo histórico imutável de lançamentos.

Componentes e responsabilidades

Componente	Responsabilidade
API NestJS	Valida contratos, recebe operações e oferece consultas, ledger, reconciliação e health checks. Pode ter várias réplicas.
Domínio	Money em centavos bigint; Wallet e políticas puras para saldo, referências, reversões e idempotência.
Serviço transacional	Compartilhado por HTTP e worker. Adquire lock da wallet e persiste todos os efeitos da decisão numa transação SQL.
PostgreSQL	Fonte de verdade: wallets, transactions, ledger, inbox e outbox, com constraints e triggers.
Workers e SQS	Consomem mensagens, reprocessam referências e publicam eventos confirmados. Loops independentes e replicáveis.

Invariante central

wallet.balance = soma dos créditos do ledger - soma dos débitos do ledger

A abertura com saldo positivo também gera lançamento. Abertura zero e LOSS não movimentam dinheiro. O saldo não é reconstruído a partir dos logs operacionais.

Uma operação, do recebimento ao evento

Etapa	O que acontece
1. Entrada	Zod valida o contrato. Valores monetários são strings com duas casas decimais. O payload financeiro canônico recebe hash SHA-256.
2. Transação SQL	Registra/deduplica inbox quando há mensagem e executa SELECT FOR UPDATE na wallet. A identidade da operação é consultada sob o lock.
3. Decisão	Uma operação terminal retorna o resultado persistido. Uma nova operação passa pelas políticas financeiras e pelas validações de referência.
4. Persistência	Atualiza wallet, insere ledger quando necessário e grava estado, resultado, inbox e outbox. Constraints deferred verificam a consistência no commit.
5. Confirmação	Somente após o commit a requisição retorna sucesso ou a mensagem recebe ack. Se o consumidor morrer antes do ack, o replay encontra o resultado persistido.
6. Publicação	Outro fluxo seleciona outbox commitada com FOR UPDATE SKIP LOCKED e envia ao SQS. A publicação não faz parte da transação financeira.

Por que a concorrência não duplica dinheiro

O lock é por wallet: duas operações sobre o mesmo saldo se serializam; wallets distintas podem avançar em paralelo. Constraints de unicidade decidem colisões de identidade também entre wallets. Não há cache em memória usado como garantia de idempotência.

A mesma key com payload divergente causa conflito. O replay terminal devolve o resultado original, inclusive o saldo histórico daquela operação. Isso é diferente de consultar o saldo atual.

Limite da garantia de eventos

Se o publisher enviar um evento e morrer antes de marcar a outbox, poderá reenviá-lo. O eventId permanece estável e consumidores devem deduplicá-lo persistentemente. A entrega é at-least-once; não se promete exactly-once no broker.

Regras financeiras e proteção no banco

Tipo	Efeito e validações principais
BET	Débito positivo, sem referência. Rejeita quando o saldo é insuficiente.
WIN	Crédito positivo. Se houver referência, deve apontar para BET no mesmo escopo.
LOSS	Valor zero; não altera saldo nem cria lançamento. Referência opcional a BET.
REFUND	Crédito que reverte integralmente uma BET processada. Uma reversão deste tipo por referência.
ROLLBACK	Inverte integralmente BET, WIN ou REFUND processado. Reversão de WIN/REFUND debita e exige saldo. Uma reversão deste tipo por referência.

Referências devem coincidir em provider, player, wallet, moeda e rodada. REFUND e ROLLBACK podem reverter a mesma referência uma vez cada, conforme a unicidade por referência/tipo adotada no enunciado.

Dinheiro exato

BRL, USD e EUR usam duas casas fixas. O domínio opera com bigint e o PostgreSQL com numeric, recebido como string pelo driver. Não há conversão monetária para Number. Entradas como 1.005 são rejeitadas: não há arredondamento implícito ou conversão cambial.

Garantias no schema

Unicidade: player/moeda, idempotency key, provider/external ID, wallet/transação no ledger e referência/tipo nas reversões processadas. CHECKs impedem valores inválidos, saldos negativos e lançamentos aritmeticamente inconsistentes.

Triggers impedem UPDATE, DELETE e TRUNCATE do ledger e mutações de transações terminais. Constraint triggers verificam saldo versus ledger e a quantidade de lançamentos exigida por operação. O usuário runtime não tem permissão para alterar o schema ou desabilitar triggers.

Essas proteções se aplicam à aplicação e ao usuário restrito. Administração privilegiada do banco requer controles próprios de acesso, backup e auditoria.

Falhas, auditoria e segurança

Referências fora de ordem

Uma referência ainda indisponível produz PENDING_REFERENCE. O worker tenta novamente com backoff de 1s a 1h, até 16 tentativas ou 24h. Estado e agendamento são rechecados dentro do lock com o relógio do banco. Uma referência que permanece ausente termina rejeitada.

Retry, DLQ e recuperação

Mensagens têm visibilidade de 60s, renovada a cada 20s. Falhas transitórias recebem backoff; mensagens inválidas/conflitos permanentes seguem para DLQ, com limite de cinco entregas. Outbox tenta novamente sem descartar eventos por número de tentativas. SIGTERM interrompe novas iterações e aguarda o trabalho em andamento.

Erros permanentes de persistência classificados só são registrados como FAILED depois do rollback confirmado, sem movimentação financeira. Erros de conexão ou commit ambíguo não viram FAILED automaticamente: uma nova entrega pode descobrir um commit já realizado.

Ledger financeiro versus logs operacionais

O ledger conserva valores, direção, saldos e vínculo com a transação. Os logs JSON usam allowlist e HMAC-SHA256 para pseudonimizar identificadores. Não serializam body, tokens, senhas, valores monetários, mensagem original de erro ou stack.

LOG_HMAC_KEY secreta e compartilhada permite correlação entre processos e reinícios. Sem configuração, cada processo usa uma chave efêmera. Logs podem se perder após commit e não substituem o ledger. Pseudonimização não elimina a necessidade de controle de acesso e retenção.

Fronteira de implantação

A configuração entregue é local: portas publicadas em loopback, credenciais públicas apenas de desenvolvimento e usuário runtime separado do administrador de migrations. A autenticação foi omitida conforme permitido pelo desafio; NODE_ENV=production bloqueia a inicialização até integrar IdP. Não é uma implantação pronta para exposição pública.

Evidências e teste de carga

Última suíte registrada em 06/11/2026: **73 testes unitários / 179 assertions** e **28 testes de integração / 225 assertions**, todos aprovados. Os logs incluídos são da execução existente, não de uma nova rodada na preparação desta entrega.

Integração com PostgreSQL e LocalStack reais: três APIs, publishers concorrentes, 50 duplicatas em paralelo, disputa de saldo, constraints, atomicidade, inbox, DLQ, referências fora de ordem e crash após commit antes de ack. Após cada teste, uma consulta independente compara todas as wallets ao ledger.

Carga local após otimização dos workers

Bun 1.3.11, Windows 11, Ryzen 7 5700X3D, 31,91 GiB de RAM; Docker 29.1.3, PostgreSQL 17.11 e LocalStack 4.10.0. Três APIs e dois workers compartilham o host. Modelo fechado, 30 warmups e 650 requisições medidas; concorrência 24, ou 50 no cenário de duplicatas.

Cenário	RPS	p50 ms	p95 ms	p99 ms
Wallet disputada	140,52	23,63	713,35	859,00
20 wallets	1.026,15	19,98	37,50	56,65
50 duplicatas	333,55	71,01	140,56	149,28

Taxa de erro: 0% nos três cenários. Conflitos registrados e retries SQL: 0. A métrica de conflitos conta falhas como timeout/deadlock, não o tempo de espera por locks. As 50 duplicatas produziram um único débito e 49 replays.

Outbox: pico amostrado de 482 pendências e 0,49s de lag. Drenagem confirmada na checagem de 5,27s após HTTP, com checagens a cada 5s; não é o instante exato da última publicação. Ao final: outbox vazia e saldo consistente com o ledger.

Leitura honesta dos resultados

São rajadas curtas, não capacidade sustentada ou desempenho AWS. O modelo fechado não corrige coordinated omission. Não houve melhora consistente no HTTP após otimizações; o p99 da wallet disputada aumentou nessa execução. O ganho observado foi na drenagem da outbox. Os JSONs históricos e LOAD_TEST.md preservam metodologia e resultados anteriores.

Como executar e navegar no projeto

Subir três APIs e dois workers

```
bun install
docker compose up --build -d --scale api=3 --scale worker=2
```

APIs em localhost:3000 a 3002; PostgreSQL em localhost:55432 e LocalStack em localhost:4566. Migrations rodam antes do runtime no Compose. Preserve o volume PostgreSQL: down -v remove os dados. O Compose não persiste LocalStack entre recriações.

Validar

```
bun run typecheck
bun run test
docker compose stop worker
bun run test:integration
docker compose start worker
bun run test:load
```

Integração exige imagem atualizada, migrations aplicadas e rede backend-challenge_default. Execute integração e carga separadamente. Integração usa portas adicionais 3101 a 3103; carga exige os workers ativos. Configurações administrativas e runtime estão descritas em LOGGING.md.

Consulta ou ação	Rota
Criar / consultar wallet	POST /wallets; GET /wallets/:id
Processar operação	POST /wagering/transactions
Consultar operação	GET /wagering/transactions/:id
Ledger / reconciliação	GET /wallets/:id/ledger; POST /wallets/:id/reconciliation
Saúde e métricas	GET /health/live; GET /health/ready; GET /metrics

Fontes e limites conhecidos

README.md: execução e enunciado original. ARCHITECTURE.md: decisões. TESTING.md e reports/logs/: evidências. LOAD_TEST.md e reports/*.json: experimentos. LOGGING.md: privacidade. src/migrations/: garantias do schema. requests.http: exemplos de uso.

A reconciliação no commit soma o histórico da wallet, com custo crescente. Não há double-entry, IdP, replay administrativo de DLQ, dashboard ou OpenTelemetry. Os testes sustentam os cenários verificados; não constituem prova absoluta de ausência de falhas.